

Day 2

JSON & API Automation

against your Day-1 API

6 hours · 3 modules · 3 labs · same repo as yesterday

Where we left off

A FastAPI server on `localhost:8000` exposing your `ProductCatalog`.
Today: **drive that server from Python**, with typed payloads.

<u>Day 1</u>	<u>Day 2 (today)</u>
server	→ typed server (Pydantic) + APIClient + CSV bulk-import → import_report.json

If you're behind: `cp -r ../checkpoints/day-2-start/. .`

Today's arc

Module	~40 min	80 min lab
1	JSON deep + Pydantic	Lab 4: Pydantic on the API
2	REST with requests	Lab 5: Build the APIClient
3	Data-driven patterns	Lab 6: CSV → API bulk import

Same `product-catalog/` repo, three new files: `models.py` (upgraded), `client.py`, `import_csv.py`.

Module 1

JSON Deep + Pydantic Validation

~40 min · then 80 min lab

JSON — the parts that bite

```
{  
  "id": 1,  
  "name": "USB-C Cable",  
  "price": 499.0,  
  "in_stock": true,  
  "tags": ["cable", "usb-c"]  
}
```

- **No trailing commas.** No comments. (Some tools accept them — the spec doesn't.)
- `true` / `false` / `null` are lowercase, unquoted.
- Numbers can be int or float but **no leading zeros, no NaN, no Infinity.**
- All keys are strings. JSON has no tuples, no sets, no dates.

Map JSON ↔ Python: `object` ↔ `dict`, `array` ↔ `list`, the rest is obvious.

Stdlib `json` — the 4 functions

```
import json

data = json.loads('{"id": 1, "name": "x"}')    # str → dict
text = json.dumps(data, indent=2)             # dict → str

with open("catalog.json") as fh:
    data = json.load(fh)                      # file → dict
with open("catalog.json", "w") as fh:
    json.dump(data, fh, indent=2)             # dict → file
```

`load` / `dump` work with file handles; `loads` / `dumps` work with strings.

The `s` is for **string**.

JSON pitfalls in real code

```
json.dumps({"price": Decimal("4.99")})  
# TypeError: Object of type Decimal is not JSON serializable  
  
json.dumps({datetime.now(): 1})  
# TypeError: keys must be str, int, float, bool or None  
  
json.dumps({"data": float("nan")})  
# Returns 'NaN' – INVALID JSON. Other languages reject it.
```

Defenses:

- Use `default=str` in `dumps` for non-trivial values
- Use Pydantic — it handles `datetime`, `Decimal`, `UUID` correctly out of the box

Why Pydantic, not raw dicts

```
# raw dict – silently wrong forever
product = {"id": "one", "name": "", "price": -50}

# Pydantic – caught at the boundary
class Product(BaseModel):
    id: int = Field(ge=1)
    name: str = Field(min_length=1)
    price: float = Field(ge=0)

Product.model_validate({"id": "one", "name": "", "price": -50})
# ValidationError:
#   id    → Input should be a valid integer
#   name  → String should have at least 1 character
#   price → Input should be ≥ 0
```

Three Pydantic models, one resource

```
class ProductBase(BaseModel):           # shared fields
    name: str
    category: str
    price: float = Field(ge=0)

class ProductCreate(ProductBase):       # POST body: caller supplies id
    id: int = Field(ge=1)

class ProductUpdate(BaseModel):         # PATCH body: all optional
    name: str | None = None
    price: float | None = Field(default=None, ge=0)
    model_config = ConfigDict(extra="forbid")

class Product(ProductBase):            # response / storage
    id: int
```

Coercion vs validation

```
ProductCreate.model_validate({  
    "id": "1",          # str → int ✓ coerced  
    "name": "Widget",  
    "category": "Misc",  
    "price": "9.50",    # str → float ✓ coerced  
    "in_stock": "true", # str → bool ✓ coerced  
})
```

Pydantic 2 is **lax by default** — strings that *look like* numbers/bools become them. This is what lets CSV → API work without preprocessing.

Need strict? `model_validate(data, strict=True)` or `Field(strict=True)`.

@field_validator for custom logic

CSV will hand us `tags="cable|usb-c"` as one string. One validator fixes it:

```
@field_validator("tags", mode="before")
@classmethod
def _split_csv_tags(cls, v):
    if isinstance(v, str):
        return [t.strip() for t in v.split("|") if t.strip()]
    return v
```

- `mode="before"` runs **before** type coercion
- `mode="after"` runs on the parsed list
- Use it for transformations the JSON spec can't express

FastAPI + Pydantic: free /docs

```
@app.post("/products", status_code=201, response_model=Product)
def create_product(payload: ProductCreate) → Product:
    return catalog.add(Product(**payload.model_dump()))
```

What you get for that one type hint:

- Request body validated against `ProductCreate` (422 on failure with field-level errors)
- Response serialised through `Product` (extra fields stripped)
- `/docs` Swagger UI populated automatically
- `/openapi.json` for codegen if you ever need a TS client



Lab 4 — Pydantic Models for the Catalog

80 min · open `labs/lab-04-pydantic-models.md`

You'll add:

- `Product`, `ProductCreate`, `ProductUpdate` in `models.py`
- `response_model=Product` on every route
- New `PATCH /products/{id}` using `ProductUpdate`
- Bad POST → **422** with structured field errors

Module 2

REST APIs with `requests`

~40 min · then 80 min lab

The four verbs you actually use

	safe?	idempotent?	typical use
GET	✓	✓	read a resource
POST	✗	✗	create / non-idempotent action
PUT	✗	✓	replace the whole resource
PATCH	✗	✗*	partial update
DELETE	✗	✓	remove

*PATCH idempotency depends on the patch shape.

Idempotent = same request, same outcome no matter how many times you send it. Matters for retries.

Status codes (the ones you'll actually see)

Range	Meaning
2xx	All good. 200 OK, 201 Created, 204 No Content
3xx	Redirect. 301 Moved, 304 Not Modified. Usually invisible.
4xx	Your fault. 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable Entity
5xx	Server's fault. 500, 502, 503, 504

Rule of thumb: **retry 5xx and network errors; never retry 4xx.**

requests — one good pattern

```
import requests

session = requests.Session()           # pooled connections
session.headers.update({"X-Trace-Id": "abc"}) # default header

response = session.post(
    "http://localhost:8000/products",
    json=payload,                        # serialises dict → JSON + sets Content-Type
    timeout=5.0,                         # ALWAYS set a timeout
)
response.raise_for_status() # raises HTTPError on 4xx/5xx
return response.json()      # parses response body
```

Without `timeout=` you risk hanging forever. Don't ship code without it.

Auth: tokens & headers

```
# Bearer token (most common)
session.headers["Authorization"] = f"Bearer {token}"

# Basic auth
requests.get(url, auth=("alice", "secret"))

# API key in header
session.headers["X-API-Key"] = key

# API key in query string (avoid – leaks into logs)
session.params = {"api_key": key}
```

Put secrets in **env vars**, never source. `os.environ["CATALOG_TOKEN"]`.

Wrap it in a class — APIClient

```
class APIClient:
    def __init__(self, base_url, *, timeout=5.0, session=None):
        self.base_url = base_url.rstrip("/")
        self._session = session or requests.Session()
        self.timeout = timeout

    @retry(times=3, delay=0.2,
           exceptions=(requests.ConnectionError, requests.Timeout))
    def _request(self, method, path, **kwargs):
        kwargs.setdefault("timeout", self.timeout)
        resp = self._session.request(method, f"{self.base_url}{path}", **kwargs)
        if not resp.ok:
            raise APIError(resp.status_code, resp.json().get("detail", resp.text))
        return resp
```

One `_request`. All other methods funnel through it.

Typed return values, not dicts

```
def list_products(self) → list[Product]:  
    data = self._request("GET", "/products").json()  
    return [Product.model_validate(row) for row in data]  
  
def create_product(self, payload: ProductCreate) → Product:  
    data = self._request("POST", "/products",  
                        json=payload.model_dump()).json()  
    return Product.model_validate(data)
```

Callers stay in Pydantic-land all the way down. The dict only exists for ~1 line inside the client.

When to retry — and when not

```
@retry(  
    times=3, delay=0.2,  
    exceptions=(requests.ConnectionError, requests.Timeout),  
)
```

Retry:

- Connection refused / DNS / timeout (network blip)
- 502 / 503 / 504 (upstream temporarily unavailable)

Don't retry:

- 400 / 422 — your payload is wrong, more tries won't fix it
- 401 / 403 — your auth is wrong
- 409 — the conflict is real



Lab 5 — Build the APIClient

80 min · open `labs/lab-05-api-client.md`

You'll build:

- `catalog/client.py` — `APIClient` + `APIError`
- Full typed CRUD via Pydantic models
- `@retry` on every HTTP call

End state: `APIClient().list_products()` returns `list[Product]`.

Module 3

Data-Driven Automation Patterns

~40 min · then 80 min lab

Chaining: output of A → input of B

```
# create then update then list
created = client.create_product(ProductCreate( ... ))
client.update_product(created.id, ProductUpdate(price=9.99))
for p in client.list_products():
    print(p.id, p.price)
```

Real automation = small clients composed into workflows.

Tomorrow's tests will mock each step independently.

Data-driven: CSV → API

```
with open("data/products.csv") as fh:
    for row_no, row in enumerate(csv.DictReader(fh), start=2):
        try:
            payload = ProductCreate.model_validate(row)
        except ValidationError as exc:
            errors.append({"row": row_no, "errors": exc.errors()})
            continue
        client.create_product(payload)
```

Three things you must keep separate:

1. **Validation errors** — row never reached the API
2. **API errors** — server rejected it (409, 422, 500...)
3. **Successes**

Collapsing them into one bucket lies to whoever reads the report.

Environment & secrets

```
# never commit secrets
import os
TOKEN = os.environ["CATALOG_TOKEN"]
BASE_URL = os.environ.get("CATALOG_BASE_URL", "http://localhost:8000")
```

For workshops: a `.env` file + `python-dotenv`.

For production: secret manager + injected env vars.

Never `print(token)` — it ends up in CI logs forever.

Pagination

```
url = "/products?page=1"
while url:
    data = client._request("GET", url).json()
    for row in data["items"]:
        yield row
    url = data.get("next")
```

Two common shapes:

1. **Cursor**: server returns `{"items": [ ... ], "next": "/products?page=2"}`
2. **Offset**: client sends `?offset=20&limit=10`

Cursor is safer (no double-reads when items shift). Most modern APIs use it.

Rate limits & backoff

```
HTTP/1.1 429 Too Many Requests  
Retry-After: 30
```

```
if resp.status_code == 429:  
    delay = float(resp.headers.get("Retry-After", 1))  
    time.sleep(delay)  
    return self._request(method, path, **kwargs)    # retry once
```

`@retry( ... )` blindly retries — for 429s, **respect `Retry-After`** instead of hammering.

Production code: exponential backoff with jitter.

The report is the product

```
{
  "summary": {"rows_read": 19, "created": 16,
    "validation_errors": 3, "api_errors": 0},
  "validation_errors": [
    {"row": 18, "input": {"name": ""},
      "errors": [{"loc": ["name"], "msg": "String should have at least 1 character"}]}
  ]
}
```

The CSV is input. The report is the **artifact you hand back to the operator** so they can fix row 18 and re-run. This report is what Day 3's tests will assert against.



Lab 6 — Bulk-Import Workflow

80 min · open `labs/lab-06-bulk-import.md`

You'll build:

- `data/products.csv` with intentionally bad rows
- `catalog/import_csv.py` runnable as `python -m catalog.import_csv`
- `import_report.json` with `created`, `validation_errors`, `api_errors`

End of Day 2 → your repo matches `checkpoints/day-3-start/`.

End of Day 2

Tomorrow: lock everything we just built under a real test suite — pytest, mocks, parametrize, HTML reports, GitHub Actions CI.

Before you leave: commit, push, take a screenshot of `import_report.json`.